Compte rendu C++

Table des matières

1 Introduction **2**

2 Présentation du projet **2**
2.1 Description fonctionnelle 2
2.2 Ce que fait concrètement le programme 2

3 Architecture et conception **2**
3.1 Organisation générale 2
3.2 Classes principales 2
3.3 Logique orientée objet 3

4 Choix techniques **3**
4.1 Structures de données 3
4.2 Utilisation de C++ et STL 3
4.3 Justification des choix 3

5 Performance et optimisation **3**
5.1 Complexité globale 3
5.2 Gestion mémoire 3
5.3 Améliorations possibles 3

6 Tests et validation **4**
6.1 Méthode de test 4
6.2 Cas limites 4
6.3 Robustesse 4

7 Difficultés rencontrées **4**

8 Conclusion **5**

1 Introduction

Ce projet s'inscrit dans le cadre du cours de C++ pour les mathématiques appliquées. L'objectif était de réaliser une simulation de dynamique moléculaire en deux dimensions, à partir d'un système de particules en interaction. On part presque de rien et on construit petit à petit toutes les briques nécessaires : une classe pour représenter une particule, un vecteur, puis une grille spatiale et enfin un univers complet capable de simuler un système physique réaliste.

L'idée de base est assez simple à formuler mais beaucoup moins simple à implémenter de manière efficace. On a un grand nombre de particules qui interagissent entre elles via un potentiel de type Lennard-Jones, et on veut suivre leur évolution au cours du temps. Le tout doit rester maniable même avec plusieurs milliers de particules.

Au-delà de la physique, c'était aussi pour nous une vraie occasion de pratiquer la conception orientée objet, la séparation interface/implémentation, l'utilisation de CMake pour gérer un projet structuré, et l'écriture de tests unitaires avec GoogleTest. On a essayé de soigner l'organisation du code parce qu'on s'est vite rendu compte qu'un projet mal structuré devient ingérable au bout d'un moment.

2 Présentation du projet

2.1 Description fonctionnelle

Le programme simule l'impact d'un disque circulaire de particules sur une membrane horizontale. Les particules de la membrane forment un réseau immobile au départ, et le disque arrive avec une vitesse initiale dirigée vers la membrane. Toutes les particules interagissent entre elles via le potentiel de Lennard-Jones, qui fait que deux particules se repoussent quand elles sont trop proches et s'attirent légèrement à moyenne distance.

À chaque pas de temps, le programme calcule les forces qui s'exercent sur chaque particule, met à jour les vitesses puis les positions, et sauvegarde l'état du système. Les sauvegardes sont écrites au format VTK, ce qui permet de visualiser ensuite la simulation dans Paraview sous forme d'animation.

2.2 Ce que fait concrètement le programme

Le déroulement est assez direct. On commence par créer un univers vide aux dimensions imposées par le sujet ($L_1 = 250$, $L_2 = 180$). On y ajoute les particules : 395 pour le disque, environ 17 000 pour la membrane. Une fois tout placé, le schéma de Störmer-Verlet prend le relais et fait avancer la simulation jusqu'à $t = 29.5$. Tous les 100 pas environ, une frame VTK est sauvegardée. À la fin, on a une série de fichiers qui permettent de revoir tout l'impact image par image.

3 Architecture et conception

3.1 Organisation générale

Le projet est organisé selon une structure CMake classique. Les déclarations sont dans `include/`, les implémentations dans `src/`, le programme de démonstration dans `demo/`, les tests unitaires dans `test/` et la documentation Doxygen dans `doc/`. Chaque sous-dossier a son propre `CMakeLists.txt`, et le `CMakeLists.txt` racine orchestre l'ensemble. C'est une organisation qu'on a vue dans plusieurs projets professionnels et qu'on a essayé de reproduire ici.

On a aussi mis en place un `.gitignore` pour ne pas pousser le dossier `build/` sur le dépôt Git, parce qu'au début on a fait l'erreur et c'est devenu vite ingérable (chemins absolus différents entre nos machines, conflits de cache CMake, etc.).

3.2 Classes principales

Le projet est articulé autour de cinq classes principales. La classe **Particule** représente une particule physique avec sa position, sa vitesse, la force qui s'exerce sur elle, sa masse et un identifiant. C'est la brique de base.

La classe **vecteur** sert à manipuler des vecteurs 3D avec les opérations classiques (addition, soustraction, produit scalaire, norme). Elle est plus légère et indépendante des autres classes.

La classe **Cellule** contient un sous-ensemble de particules et connaît l'index de ses cellules voisines dans la grille. C'est la pièce centrale de l'optimisation par cellules liées : au lieu de comparer chaque particule à toutes les autres, on ne regarde que celles qui sont dans des cellules voisines.

La classe **Grille** regroupe toutes les cellules dans un tableau 2D et calcule les voisinages au démarrage. Elle gère aussi l'indexation row-major.

Enfin, **Univers** orchestre toute la simulation : il contient une grille, gère le calcul des forces, l'intégration temporelle, les conditions aux bords et l'énergie cinétique. C'est l'interface principale qu'utilise le programme de démonstration.

3.3 Logique orientée objet

On a essayé d'avoir une vraie séparation des responsabilités. Une particule ne sait rien de la grille, une cellule ne sait rien de la physique, et Univers est le seul à connaître toutes les briques pour les coordonner. Au début on avait tout entassé dans Univers, ce qui rendait le fichier énorme et difficile à modifier sans casser autre chose. On a ensuite séparé proprement.

Une autre chose qu'on a soignée, c'est la séparation entre les fichiers `.hxx` (déclarations) et `.cxx` (implémentations). Au départ on avait tout mis en inline dans les headers, ce qui marche mais a deux gros défauts : à chaque modification, tous les fichiers qui incluent le header doivent être recompilés, et le header devient illisible. La séparation rend le code plus modulaire et accélère beaucoup la compilation incrémentale.

4 Choix techniques

4.1 Structures de données

Pour stocker les particules dans une cellule, on a choisi un `std::vector<Particule>`. C'était le choix le plus naturel : accès indexé en temps constant, ajout en fin de vecteur amorti en $O(1)$, et compatibilité parfaite avec les boucles range-based for.

Pour la grille, on a aussi utilisé un `std::vector<Cellule>` avec un indexage row-major (`index = x + nx * y`) plutôt qu'un `std::vector<std::vector<Cellule»`. C'est plus efficace en mémoire (un seul bloc contigu) et l'accès reste simple grâce à une méthode `getIndex(x, y)` qui fait la conversion.

4.2 Utilisation de C++ et STL

On a utilisé pas mal de fonctionnalités modernes du C++. Les `enum class` pour la catégorie de particule, qui sont plus sûres que les enum classiques. Les boucles range-based for partout où c'était possible parce que c'est beaucoup plus lisible. `std::sqrt`, `std::pow` de `<cmath>` pour les calculs. `std::ofstream` pour l'écriture des fichiers VTK. `std::filesystem::create_directory` pour créer le dossier `frames/` s'il n'existe pas. `std::chrono` pour mesurer le temps d'exécution.

On a aussi utilisé les itérateurs avec `erase()` pour la condition d'absorption, parce qu'effacer un élément en cours d'itération est un cas piégeux qu'il fallait gérer correctement (l'itérateur retourné par `erase` pointe vers l'élément suivant, c'est ce qui permet de continuer la boucle sans bug).

4.3 Justification des choix

Pour le calcul des forces, on est passé d'une approche naïve en $O(N^2)$ à la méthode des cellules liées. L'idée c'est qu'avec un rayon de coupure r_{cut}, une particule n'interagit qu'avec celles situées à moins de r_{cut}. Si on découpe l'espace en cellules de côté r_{cut}, alors les particules à moins de r_{cut} sont forcément dans la même cellule ou dans une cellule voisine. On passe donc de $O(N^2)$ à $O(N)$ environ, ce qui change tout pour 17 000 particules.

Pour le schéma d'intégration temporelle, on a choisi Störmer-Verlet plutôt qu'Euler. Verlet est un schéma symplectique d'ordre 2, ce qui veut dire qu'il conserve l'énergie sur le long terme contrairement à Euler qui dérive vite. Pour une simulation moléculaire qui dure plusieurs dizaines de milliers de pas, c'est essentiel.

5 Performance et optimisation

5.1 Complexité globale

Avec la méthode des cellules liées, le calcul des forces est en $O(N)$ par pas de temps, en supposant une densité de particules à peu près uniforme. Le calcul des conditions aux bords est en $O(N)$. La mise à jour des positions et des vitesses est en $O(N)$. Globalement, un pas de temps est en $O(N)$.

Sur la simulation complète avec environ 17 600 particules et 59 000 pas de temps, on est sur de l'ordre de 10^9 opérations de calcul de force au total. C'est lourd mais ça reste faisable sur un PC portable moderne en quelques dizaines de minutes.

5.2 Gestion mémoire

On n'a pas eu de soucis particuliers de gestion mémoire. On utilise principalement des `std::vector` qui gèrent automatiquement leur allocation. Pas de `new`/`delete` manuel, donc pas de fuite à craindre.

Le seul point un peu délicat c'est dans `maj_cellules` qui copie toutes les particules dans un vecteur temporaire, vide les cellules, puis redistribue. Ce n'est pas optimal en mémoire (on alloue un vecteur de la taille de tout le système à chaque pas) mais c'est simple et ça marche. Pour une vraie optimisation il faudrait ne déplacer que les particules qui ont changé de cellule.

5.3 Améliorations possibles

Le profiling avec gprof a confirmé que la fonction `all_forces` est largement la plus coûteuse, ce qui est attendu vu le nombre d'interactions à calculer. Plusieurs pistes d'optimisation sont envisageables.

D'abord, paralléliser la boucle externe du calcul des forces avec OpenMP serait probablement le gain le plus simple à obtenir. Les calculs sont indépendants entre cellules suffisamment éloignées.

Ensuite, la fonction `maj_cellules` pourrait être réécrite pour ne déplacer que les particules ayant traversé une frontière de cellule, au lieu de tout reconstruire. Sur un pas de temps typique, très peu de particules changent de cellule.

Enfin, on pourrait éviter certains `std::pow(x, 6)` en remplaçant par des multiplications successives, qui sont en pratique plus rapides.

On a aussi joué sur les options de compilation. Au début on compilait sans optimisation et c'était extrêmement lent. Avec `-O3 -march=native`, le gain est de l'ordre d'un facteur 5 à 10. On a aussi mis en place un mode `Profile` dans CMake qui active `-pg` pour générer le rapport gprof, ce qui nous a permis d'identifier les fonctions chaudes.

6 Tests et validation

6.1 Méthode de test

On a utilisé GoogleTest pour écrire des tests unitaires sur chaque classe principale. La récupération de GoogleTest est faite directement par CMake via `FetchContent`, ce qui évite à l'utilisateur d'avoir à installer la bibliothèque manuellement. Les tests sont compilés en exécutable séparé et lancés via `ctest`.

On a écrit des tests pour :

- La classe `Particule` : constructeurs, getters, setters, calcul de la force d'interaction (avec un cas où les particules sont trop proches pour vérifier qu'on évite la division par zéro).
- La classe `vecteur` : opérations algébriques, norme, produit scalaire.
- La classe `Cellule` : ajout et suppression de particules, gestion des voisins.
- La classe `Grille` : indexation, position des centres, initialisation des voisinages (4 voisins en coin, 6 sur bord, 9 au centre).
- La classe `Univers` : ajout de particules, mise à jour des vitesses, réflexion aux bords, calcul de l'énergie cinétique, rescaling des vitesses.

6.2 Cas limites

On a particulièrement fait attention à quelques cas piégeux. Le premier c'est la division par zéro dans le calcul de la force quand deux particules sont quasi-superposées : on a ajouté un seuil minimal en dessous duquel la force n'est pas calculée. Le deuxième c'est la suppression d'éléments dans un `std::vector` pendant qu'on l'itère, dans `appliquer_absorption`. Il faut absolument utiliser le retour de `erase` comme nouveau itérateur, sinon on a un comportement indéfini. Le troisième c'est la gestion des bords du domaine pour la grille : une particule à la frontière exacte ne doit pas tomber dans une cellule inexistante.

6.3 Robustesse

On a aussi ajouté un check de NaN dans la boucle principale du Stormer-Verlet. Si à un moment une particule prend une valeur `NaN` (typiquement à cause d'une force trop intense quand deux particules se rapprochent énormément), la simulation s'arrête proprement avec un message d'erreur. C'est arrivé plusieurs fois pendant le développement, et sans ce garde-fou on pouvait laisser tourner des heures avant de réaliser que les résultats étaient cassés.

On a aussi mis une saturation de la force d'interaction à ± 5000. Ça casse un peu la physique exacte mais ça évite que la simulation explose numériquement aux configurations très denses. C'est un compromis pragmatique.

7 Difficultés rencontrées

La plus grosse difficulté au début, c'était sans doute la gestion du projet en équipe avec Git. On ne s'était pas mis d'accord sur une convention de nommage des fichiers (`.cpp` vs `.cxx`, `.hpp` vs `.hxx`), et on s'est retrouvés à chaque `git pull` avec des conflits parce qu'on n'avait pas la même structure. On a aussi eu le coup classique du dossier `build/` versionné qui contient des chemins absolus différents selon la machine. À un moment CMake refusait carrément de continuer à cause d'un cache hérité d'un autre PC. On a fini par ajouter un `.gitignore` propre et dé-tracker tous les fichiers générés, ce qui aurait dû être fait dès le début.

Une autre difficulté était la compréhension fine du potentiel de Lennard-Jones et de sa force associée. Le passage de la formule du potentiel à celle de la force (qui est son gradient) demande un peu de calcul, et il faut faire très attention aux signes pour ne pas inverser une attraction et une répulsion. On s'en est rendu compte quand on a vu nos premières simulations exploser au lieu de coller.

La calibration du pas de temps a aussi pris du temps. Avec un `dt` trop grand, le schéma de Verlet

devient instable et l'énergie diverge. Avec un `dt` trop petit, la simulation est correcte mais prend des heures. Le sujet imposait $\delta t = 0.0005$ ce qui est plutôt petit, et nous a effectivement protégé des instabilités numériques.

Enfin, on a passé un certain temps à comprendre pourquoi `all_forces` appelait automatiquement la gravité et la réflexion en interne. C'était un héritage d'une première version où on voulait que tout soit "automatique", mais ça empêchait de découpler proprement les contributions. On a fini par séparer les responsabilités : `all_forces` ne calcule que les forces inter-particules, et c'est à l'appelant d'ajouter ce qu'il veut (gravité, conditions aux bords). Plus modulaire, mais ça a cassé temporairement la démo qu'il a fallu adapter.

8 Conclusion

Au final, on a un programme qui fait ce qui était demandé : la simulation tourne, produit les frames VTK, et les résultats visualisés dans Paraview montrent bien l'impact du disque sur la membrane. La structure du code nous semble propre, les tests passent, la documentation Doxygen se génère correctement, et le profiling permet d'identifier les fonctions critiques. C'est plutôt satisfaisant.

Au-delà du résultat lui-même, c'est probablement le projet où on a le plus appris sur les outils de développement C++. CMake, Doxygen, Git, GoogleTest, gprof, Paraview, gdb : tout ça nous était plus ou moins familier mais on n'avait jamais eu à tout faire fonctionner ensemble dans un vrai projet. C'est un autre niveau de complexité par rapport à un TP isolé.

Plusieurs pistes d'amélioration restent ouvertes. Côté physique, on pourrait étendre le code à la 3D, ce qui demanderait surtout de généraliser la grille en 3D et d'adapter le calcul des voisinages (27 voisins au lieu de 9). Côté performance, paralléliser `all_forces` avec OpenMP est probablement le gain le plus simple. Côté architecture, séparer les classes `Cellule` et `Grille` dans leurs propres fichiers serait plus propre, même si dans notre cas la séparation actuelle reste lisible.

Si on devait recommencer, on partirait probablement avec une architecture mieux pensée dès le début (séparation déclaration/implémentation, `.gitignore` dès le commit initial, conventions de nommage figées dans un `README`). On a beaucoup retravaillé le code en cours de route, et c'est un coût qu'on aurait pu éviter en posant les bases plus solidement au départ. C'est sans doute la principale leçon qu'on retient.